

Tutorial7

November 18, 2025

Name: __HAN Zhuo__

EID: __zhuohan3__

1 CS5489 - Tutorial 7

1.1 Dimensionality Reduction and Face Recognition

In this tutorial you will use linear and non-linear dimensionality reduction on face images, and then train a classifier for face recognition. In the second part, you will then use dimensionality reduction for de-noising.

First we need to initialize Python. Run the below cell.

```
[1]: %matplotlib inline
import matplotlib_inline # setup output image format
matplotlib_inline.backend_inline.set_matplotlib_formats('svg')
import matplotlib.pyplot as plt
import matplotlib
from numpy import *
from sklearn import *
import glob
import os
random.seed(100)
rbow = plt.get_cmap('rainbow')
```

1.2 Loading Data and Pre-processing

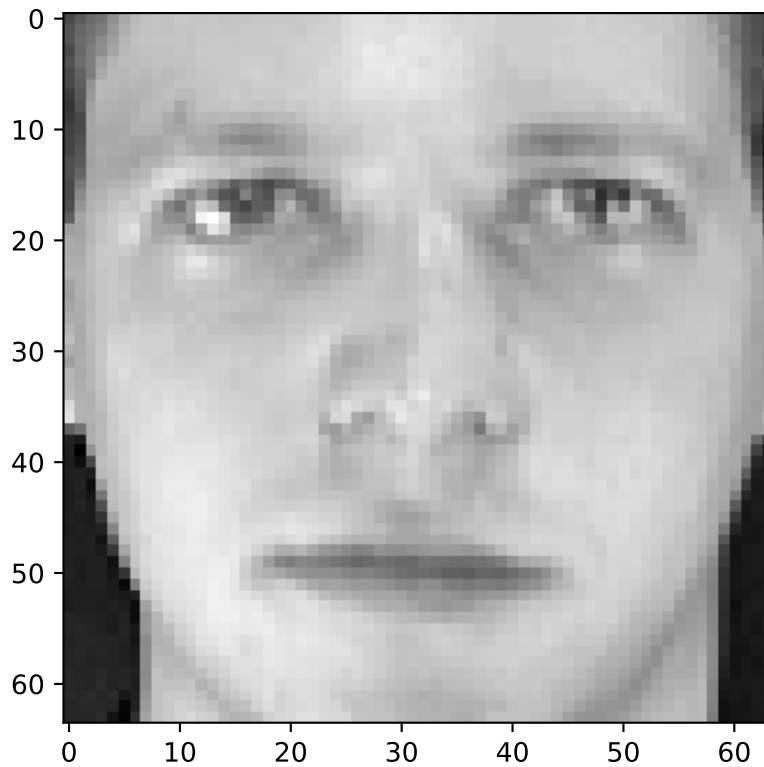
We first need to load the images. Download `olivetti_py3.pkz` from Canvas, and place it in in the same directory as this ipynb file. *DO NOT UNZIP IT*. Then run the following cell to load the images.

```
[2]: oli = datasets.fetch_olivetti_faces(data_home="./")
X = oli.data.astype('float')
Y = oli.target
img = oli.images
imgsize = oli.images[0].shape
```

Each image is a 64x64 array of pixel values, resulting in a 4096 dimensional vector. Run the below code to show an example:

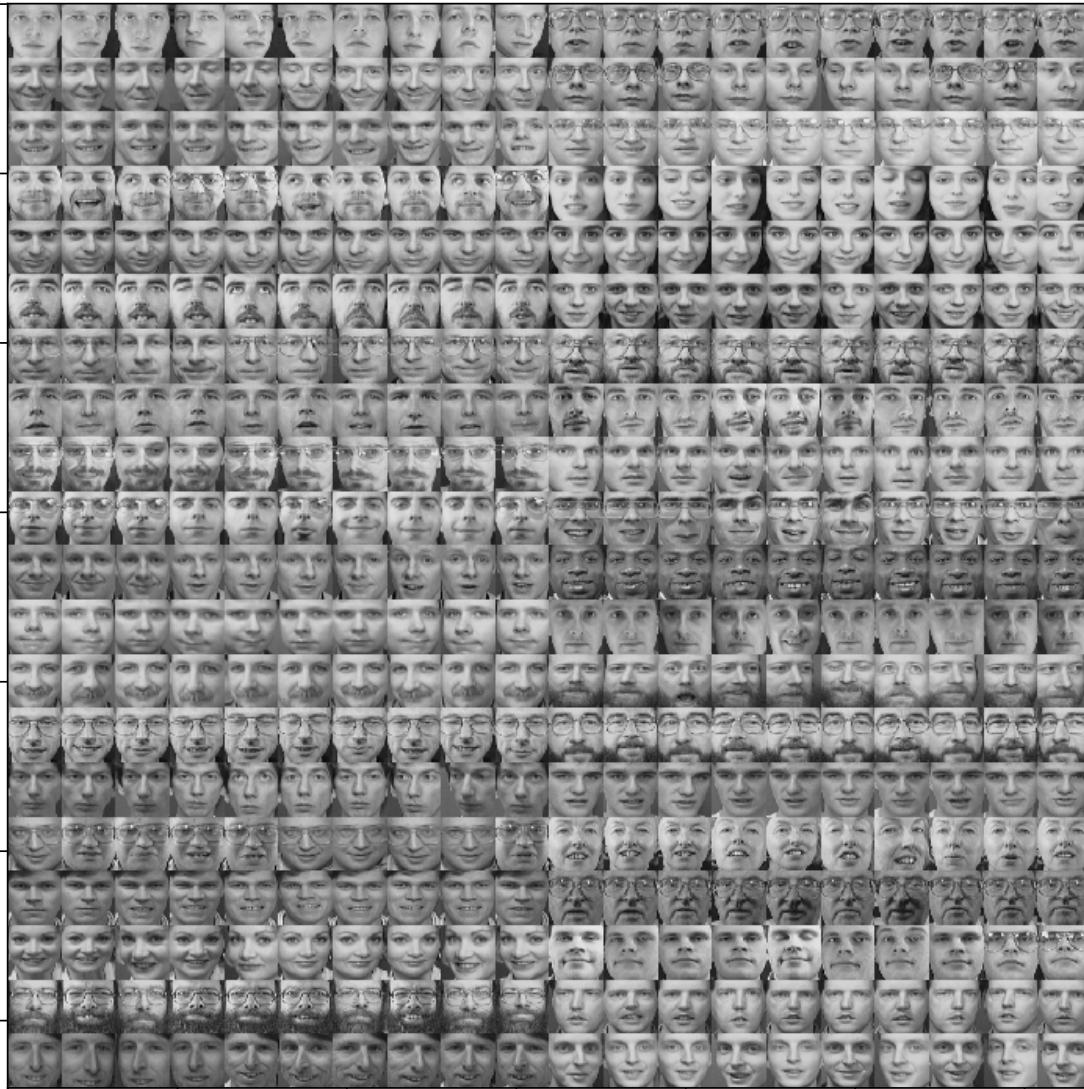
```
[3]: print(img[0].shape)
plt.imshow(img[0], cmap='gray', interpolation='nearest')
plt.show()
```

(64, 64)



Run the below code to show all the images!

```
[4]: tmp = []
for i in range(0,400,20):
    tmp.append( hstack(img[i:i+20]) )
allimg = vstack(tmp)
plt.figure(figsize=(9,9))
plt.imshow(allimg, cmap='gray', interpolation='nearest')
plt.gca().xaxis.set_ticklabels([])
plt.gca().yaxis.set_ticklabels([])
plt.show()
```



Each person is considered as one class, and there are 10 images for each class. In total there are 40 classes (people). The data is already vectorized and put into the matrix **X**. The class labels are in vector **Y**. Now we split the data into training and testing sets.

```
[5]: # randomly split data into 80% train and 20% test set
trainX, testX, trainY, testY = \
    model_selection.train_test_split(X, Y,
    train_size=0.80, test_size=0.20, random_state=4487)

print(trainX.shape)
print(testX.shape)
```

```
(320, 4096)
```

```
(80, 4096)
```

2 Linear Dimensionality Reduction - PCA

The dimension of the data is too large (4096) so learning classifiers will take a long time. Instead, our strategy is to use PCA to reduce the dimension first and then use the PCA weights as the representation for each image. Run PCA on the data using 9 principal components.

```
[6]: ### INSERT YOUR CODE HERE
from sklearn.decomposition import PCA

n_components = 9
pca = PCA(n_components=n_components, whiten=True, random_state=0)
pca.fit(trainX)
trainX_pca = pca.transform(trainX)
testX_pca = pca.transform(testX)
```

```
[8]: print("Original dim:", trainX.shape[1])
print("Reduced dim:", trainX_pca.shape[1])
print("trainX_pca shape:", trainX_pca.shape)
print("testX_pca shape :", testX_pca.shape)
```

```
Original dim: 4096
Reduced dim: 9
trainX_pca shape: (320, 9)
testX_pca shape : (80, 9)
```

The below function will plot the basis vectors of PCA. Run the next 2 cells to view the PCs.

```
[7]: def plot_basis(model, imgsize):
    cname = model.__class__.__name__
    if cname == 'LDA':
        KK = model.n_components
        comps = model.coef_
        mn = None
    elif cname == 'PCA':
        KK = model.n_components_
        comps = model.components_
        mn = model.mean_
    elif cname == 'NMF':
        KK = model.n_components_
        comps = model.components_
        mn = None
    K = KK
    if mn is not None:
        K += 1
    nr = int(ceil(K/5.0))
    sind = 1

    #vmin = comps.flatten().min()
```

```

#vmax = comps.flatten().max()

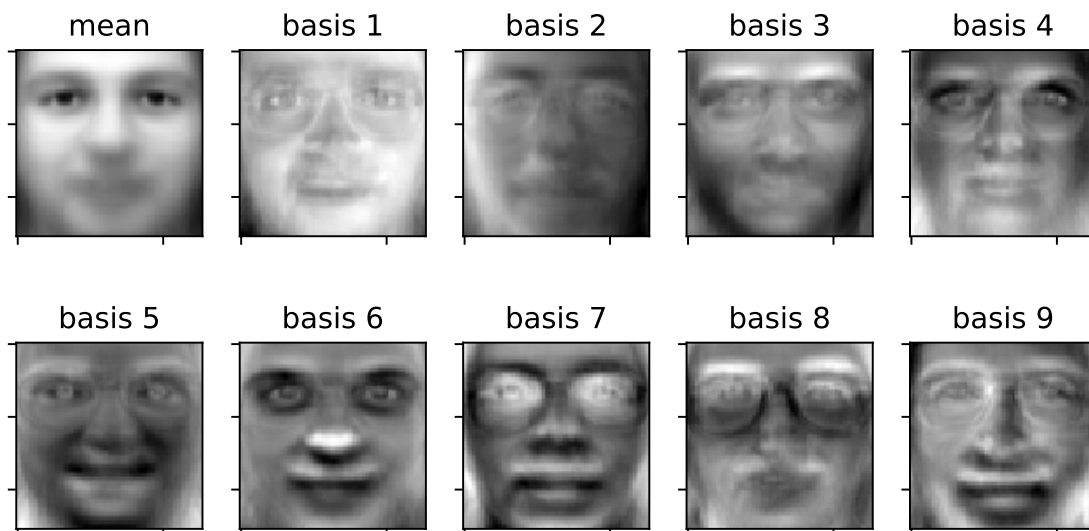
# plot the mean
pcfig = plt.figure(figsize=(8,nr*2))
if mn is not None:
    plt.subplot(nr,5,sind)
    plt.imshow(mn.reshape(imgsize), interpolation='nearest')
    plt.title("mean")
    plt.gray()
    plt.gca().xaxis.set_ticklabels([])
    plt.gca().yaxis.set_ticklabels([])
    sind += 1
# plot the components
for j in range(0,KK):
    plt.subplot(nr,5,sind)
    v = comps[j,:]
    I = v.reshape(imgsize)
    plt.imshow(I, interpolation='nearest')
    plt.gray()
    plt.title("basis " + str(j+1))
    plt.gca().xaxis.set_ticklabels([])
    plt.gca().yaxis.set_ticklabels([])
    sind += 1

```

```

[10]: # run the function
plot_basis(pca, imgsize)

```



What do the basis images look like? Do some basis images correspond to particular facial features?
 - INSERT YOUR ANSWER HERE - The basis images look like blurry faces. They are

not single persons, but patterns showing how pixel values tend to vary across the dataset. - Yes, some basis images do capture particular faicial features. Several of these bases clearly correspond to particular facial features. For example, some components mainly change the brightness of the whole face, some emphasize the eye region or the presence of glasses, and others highlight the mouth and cheeks.

2.1 Face Recognition

Now train a *linear* classifier to do the face recognition. Use the calculated PCA representation as the new set of inputs. Use cross-validation to set the hyperparameters of the classifier. *You do not need to do cross-validation for the number of components.* Calculate the average training and testing accuracies. Remember to transform the test data into the PCA representation too!

NOTE: It is **NOT** recommended to use `svm.SVC` for the training. This is because the default SVC implements 1-vs-1 multi-class classification, which means it will calculate $40 \times 39 = 1560$ classifiers! All other classifier should be fine since they do 1-vs-all multi-class classification, e.g., `LogisticRegressionCV`, `LinearSVC`, etc. If you want to use `svm.SVC`, make sure to use the `OneVsRestClassifier` as shown in Lecture 3.

```
[8]: import warnings

warnings.filterwarnings(
    "ignore",
    category=UserWarning,
    message="The number of unique classes is greater than 50% of the number of_\n\
↪samples.*"
)
warnings.filterwarnings(
    "ignore",
    category=UserWarning,
    message="The least populated class in y has only 5 members, which is less_\n\
↪than n_splits=10*"
)
warnings.filterwarnings(
    "ignore",
    category=FutureWarning,
    message="'multi_class' was deprecated in version 1.5 and will be removed in_\n\
↪1.8. From then on, it will always use 'multinomial'. Leave it to its default_\n\
↪value to avoid this warning.*"
)
warnings.filterwarnings(
    "ignore",
    category=UserWarning,
    message="Maximum number of iterations *"
)
```

```
[9]: ### INSERT YOUR CODE HERE
from sklearn.linear_model import LogisticRegressionCV, LogisticRegression
```

```

from sklearn.metrics import accuracy_score

lr_cv = LogisticRegressionCV(Cs=1000, cv=10, multi_class='multinomial',
    ↪max_iter=5000, n_jobs=-1, solver='lbfgs')
lr_cv.fit(trainX_pca, trainY)

train_pred = lr_cv.predict(trainX_pca)
test_pred = lr_cv.predict(testX_pca)

train_acc = accuracy_score(trainY, train_pred)
test_acc = accuracy_score(testY, test_pred)

print("Training accuracy:", train_acc)
print("Testing accuracy:", test_acc)

```

Training accuracy: 0.9625

Testing accuracy: 0.7625

2.2 Finding the best number of components

Now try a range of number of components for PCA to get the best test accuracy. Train a classifier for each one and see which dimension gives the best testing accuracy. Make a plot of PCA dimension vs. test accuracy.

```

[13]: ### INSERT YOUR CODE HERE
import numpy as np

dims = range(1, 101)
train_accs = []
test_accs = []

try:
    C_attr = getattr(lr_cv, "C_", 1.0)
    if np.ndim(C_attr) > 0:
        base_C = float(np.ravel(C_attr)[0])
    else:
        base_C = float(C_attr)
except NameError:
    base_C = 1.0

best_test_acc = -1.0
best_dim = None
best_pca = None
best_clf = None

for d in dims:
    pca_d = PCA(n_components=d, whiten=True, random_state=0)
    pca_d.fit(trainX)

```

```

trainX_pca_d = pca_d.transform(trainX)
testX_pca_d = pca_d.transform(testX)

clf = LogisticRegression(
    C=base_C,
    multi_class='multinomial',
    max_iter=5000,
    solver='lbfgs',
    n_jobs=-1
)
clf.fit(trainX_pca_d, trainY)

train_pred = clf.predict(trainX_pca_d)
test_pred = clf.predict(testX_pca_d)
this_train_acc = accuracy_score(trainY, train_pred)
this_test_acc = accuracy_score(testY, test_pred)

train_accs.append(this_train_acc)
test_accs.append(this_test_acc)
if this_test_acc > best_test_acc:
    best_test_acc = this_test_acc
    best_dim = d
    best_pca = pca_d
    best_clf = clf
best_idx = int(np.argmax(test_accs))
print("Best dim =", dims[best_idx],
      "with test accuracy = {:.3f}".format(test_accs[best_idx]))

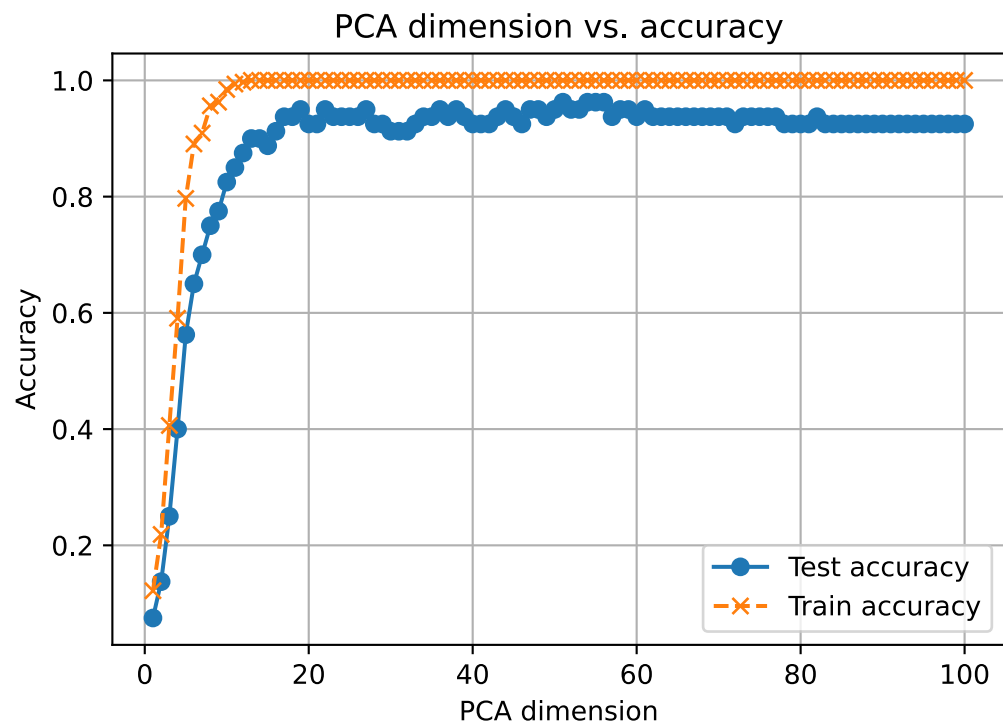
```

Best dim = 51 with test accuracy = 0.963

```

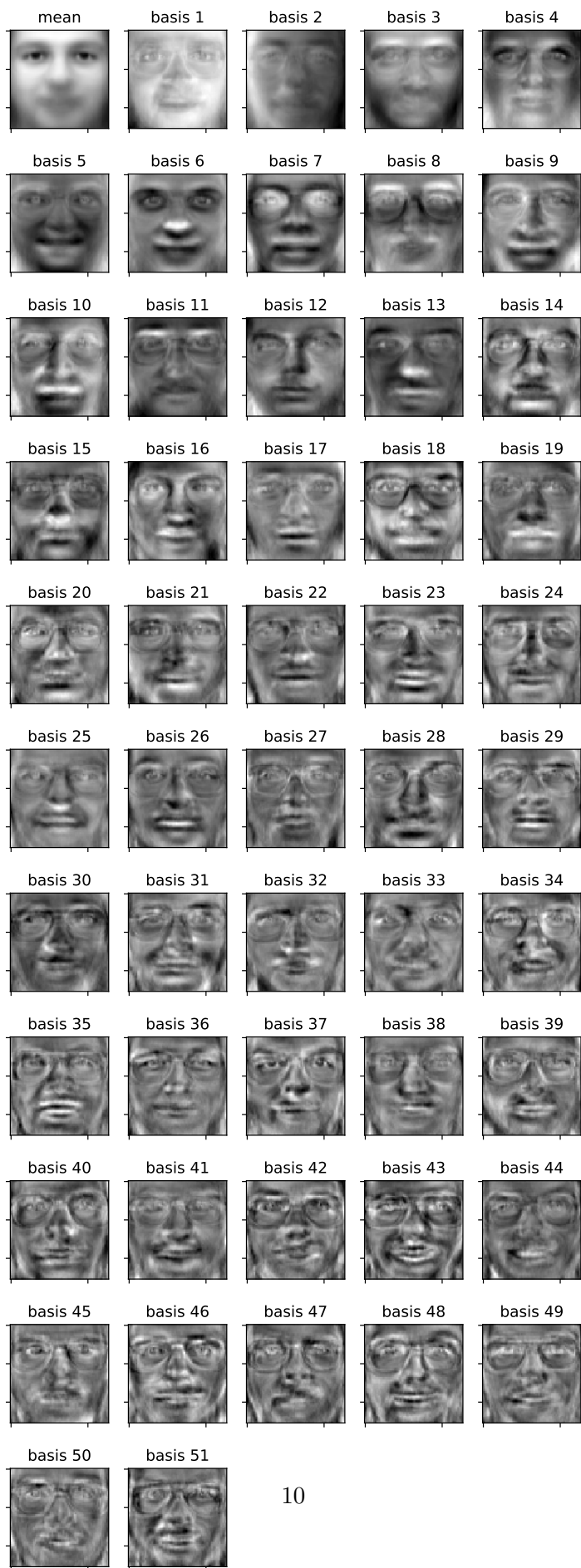
[14]: plt.figure(figsize=(6,4))
plt.plot(dims, test_accs, marker='o', label='Test accuracy')
plt.plot(dims, train_accs, marker='x', linestyle='--', label='Train accuracy')
plt.xlabel('PCA dimension')
plt.ylabel('Accuracy')
plt.title('PCA dimension vs. accuracy')
plt.legend()
plt.grid(True)
plt.show()

```



```
[15]: print(best_pca.n_components)
      plot_basis(best_pca, imgsize)
```

51



What is the best number of components? View the basis images to see what they look like - **INSERT YOUR ANSWER HERE** - From the curve and the printout, the best test accuracy is achieved at 51 PCA components, under which the test accuracy is 0.963. - From the 51-dimensional PCA model, the basis images are typical eigenfaces. The mean image is a smooth, average-looking face. The first few bases mainly capture global properties such as overall illumination and coarse face shape. As the component index increases, the bases start to emphasize local facial regions: some highlight the eye area or glasses, others focus on the mouth and cheeks (smile or neutral), eyebrows, or nose shadow. By the time we reach around 50 components, many bases correspond to subtle variations in expression and fine details rather than completely new structures, so they look like different “modes” of faces (smiling, frowning, different lighting) rather than individual people.

3 Linear Dimensionality Reduction - NMF

Now we will repeat the experiment using non-negative matrix factorization instead of PCA. Perform NMF with 9 components and visualize the basis images.

```
[10]: ### INSERT YOUR CODE HERE
from sklearn.decomposition import NMF
import matplotlib.pyplot as plt
import numpy as np

n_components = 9

nmf = NMF(
    n_components=n_components,
    init='random',
    random_state=0,
    max_iter=500
)
nmf.fit(trainX)

trainX_nmf = nmf.transform(trainX)
testX_nmf = nmf.transform(testX)

print("trainX_nmf shape:", trainX_nmf.shape)
print("testX_nmf shape :", testX_nmf.shape)

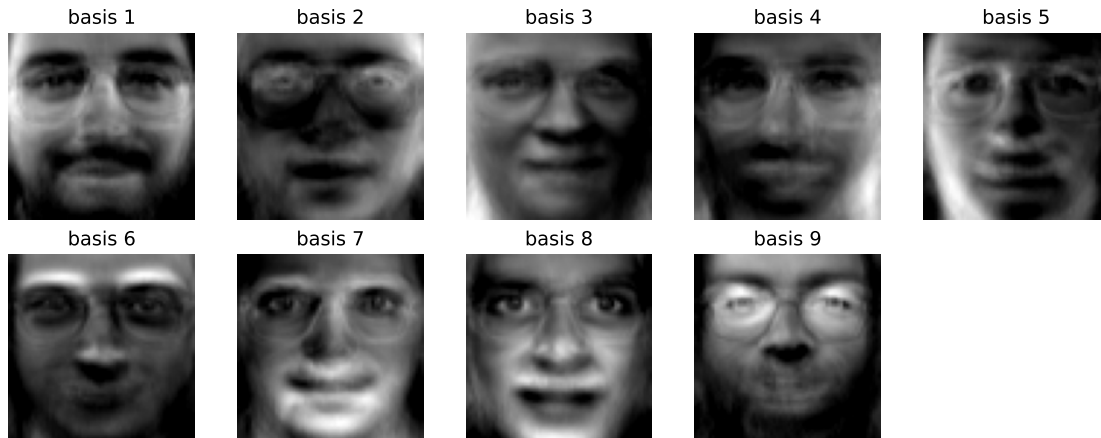
h, w = imgsize
plt.figure(figsize=(10, 4))

for i in range(n_components):
    plt.subplot(2, 5, i + 1)
    plt.imshow(nmf.components_[i].reshape(h, w), cmap='gray')
    plt.title(f"basis {i+1}")
    plt.axis('off')
```

```
plt.tight_layout()
plt.show()
```

```
trainX_nmf shape: (320, 9)
```

```
testX_nmf shape : (80, 9)
```



What do the basis images look like? What is the main difference with the PCA basis images? - **INSERT YOUR ANSWER HERE** - The NMF basis images are still blurry, but they are more localized compared to the previous style. Many bases mainly highlight certain regions (eyes/glasses, mouth, nose, cheeks), while the rest of the face stays very dark. Each basis looks like an additive “patch of a face” rather than a full face with both bright and dark lobes. - The main difference from the PCA bases is that NMF bases are non-negative and additive. PCA eigenfaces have both positive and negative values, so they look like global contrast patterns over the whole face (bright vs dark regions), whereas the NMF bases tend to be more localized components that combine additively to reconstruct a face, making them more “parts-based” and interpretable.

3.1 Finding the best number of components

Now find the number of components that gives the best test accuracy. Use the same type of classifier that you used in the previous experiment. Use cross-validation to select the hyperparameters of the classifier. You do not need to do cross-validation for the number of components.

```
[17]: ### INSERT YOUR CODE HERE
dims = range(1, 101)
train_accs_nmf = []
test_accs_nmf = []

# try:
#     C_attr = getattr(lr_cv, "C_", 1.0)
#     if np.ndim(C_attr) > 0:
#         base_C = float(np.ravel(C_attr)[0])
```

```

#     else:
#         base_C = float(C_attr)
# except NameError:
base_C = 1.0

best_dim_nmf = None
best_test_acc_nmf = -1.0
best_nmf = None
best_clf_nmf = None

for d in dims:
    # print(d)
    nmf_d = NMF(
        n_components=d,
        init='nndsvd',
        random_state=0,
        max_iter=500,
        tol=1e-3
    )
    trainX_nmf_d = nmf_d.fit_transform(trainX)
    testX_nmf_d = nmf_d.transform(testX)

    clf = LogisticRegression(
        C=base_C,
        multi_class='multinomial',
        max_iter=5000,
        solver='lbfgs',
        n_jobs=-1
    )
    clf.fit(trainX_nmf_d, trainY)

    train_pred = clf.predict(trainX_nmf_d)
    test_pred = clf.predict(testX_nmf_d)

    train_acc = accuracy_score(trainY, train_pred)
    test_acc = accuracy_score(testY, test_pred)

    train_accs_nmf.append(train_acc)
    test_accs_nmf.append(test_acc)

    if test_acc > best_test_acc_nmf:
        best_test_acc_nmf = test_acc
        best_dim_nmf = d
        best_nmf = nmf_d
        best_clf_nmf = clf

print("Best NMF dim =", best_dim_nmf,

```

```
"with test accuracy = {:.3f}".format(best_test_acc_nmf))
```

```
/home/hanzhuo/miniconda3/envs/hz/lib/python3.12/site-  
packages/joblib/externals/loky/process_executor.py:782: UserWarning: A worker  
stopped while some jobs were given to the executor. This can be caused by a too  
short worker timeout or by a memory leak.
```

```
warnings.warn(  

```

```
Best NMF dim = 95 with test accuracy = 0.887
```

*Which number of components give the best test result? How does the accuracy compare to the best PCA result? Why is NMF or PCA better? - **INSERT YOUR ANSWER HERE*** - With NMF, the best test result is obtained with 95 components, giving a test accuracy of about 0.887. With PCA, the best result was at 51 components, with test accuracy about 0.963, so PCA is still clearly better in this setting. - PCA finds orthogonal directions that maximize variance and can use both positive and negative values, so a relatively small number of components (around 50) already captures most of the global structure of faces and is very discriminative for the 40-class task. NMF, constrained to non-negative, “parts-based” bases, needs many more components (at 95) to reach a similar representational power and still does not reach the PCA accuracy. Its components are more interpretable (eyes, mouth, etc.), but this constraint makes it less efficient than PCA for this high-dimensional face recognition problem.

4 Non-Linear Dimensionality Reduction - KPCA

Now we will use KPCA to perform non-linear dimensionality reduction on the data before training the classifier. Run KPCA on the data using 10 principal components. Use the RBF kernel with gamma=0.001.

```
[11]: ### INSERT YOUR CODE HERE ###  
from sklearn.decomposition import KernelPCA  
  
kpca = KernelPCA(  
    n_components=10,  
    kernel='rbf',  
    gamma=0.001,  
    fit_inverse_transform=False,  
    random_state=0  
)  
  
trainX_kpca = kpca.fit_transform(trainX)  
testX_kpca = kpca.transform(testX)  
  
print("trainX_kpca shape:", trainX_kpca.shape)  
print("testX_kpca shape :", testX_kpca.shape)
```

```
trainX_kpca shape: (320, 10)
```

```
testX_kpca shape : (80, 10)
```

The below function will plot the basis vectors of KPCA. Run the next 2 cells to view the PCs. The

kernel PCs are a combination of similarities to points in the training set. The PCs are visualized by showing the top 5 positive and negative training examples, along with their coefficient α_i .

```
[12]: def plot_kbasis(model, imgsize, X):
    KK = model.n_components
    alphas = model.eigenvectors_.T
    minmax = 5

    py = KK
    px = minmax*2
    for i in range(KK):
        # sort alphas
        inds = argsort(alphas[i,:])

        myi = r_[arange(-1,-minmax-1,-1), arange(minmax-1,-1,-1)]
        myinds = inds[myi]

        for j,jj in enumerate(myinds):
            plt.subplot(py,px,(j+1)+i*px)
            plt.imshow(X[jj,:].reshape(imgsize), interpolation='nearest')
            plt.gray()
            if alphas[i,jj]<0:
                mycol = 'b'
            else:
                mycol = 'r'
            plt.title("{:.3f}".format(alphas[i,jj]), fontsize=7, color=mycol)
            if (j==0):
                plt.ylabel('PC' + str(i+1))
            plt.xticks([])
            plt.yticks([])

[13]: # run the function
plt.figure(figsize=(10,12))
plot_kbasis(kpca, imgsize, trainX)
```



What is the interpretation for the KPCA basis? What kind of faces do some of the PCs prefer? - **INSERT YOUR ANSWER HERE** - For KPCA the “basis” is still a set of principal directions, but now in a non-linear feature space defined by the RBF kernel. The plot shows, for each PC, those training faces that get very positive (red) or very negative (blue) scores along that component. From the figure we see that different PCs prefer different types of faces: Some PCs separate smiling vs. neutral faces (positive side full of big smiles, negative side more neutral expressions); Some PCs distinguish faces with glasses vs. without, or emphasize strong shadows around the eyes; Others seem to capture lighting and contrast (bright, evenly lit faces on one side, darker or high-contrast

faces on the other), or slightly different poses / aspect ratios. So each KPCA component corresponds to a non-linear direction that prefers faces sharing certain attributes (expression, glasses, illumination, etc.), and penalizes faces with the opposite characteristics.

4.1 Finding the best kernel and best number of components

Now try different kernels (poly, RBF, etc), kernel parameters, and number of components to get the best test accuracy. Train a classifier for each one and see which dimension gives the best testing accuracy. Make a plots of number of components vs. test accuracy.

```
[14]: ### INSERT YOUR CODE HERE
from tqdm.auto import tqdm

dims = range(1, 101)
kernel_settings = [
    ("rbf", {"gamma": 1e-3}),
    ("rbf", {"gamma": 1e-4}),
    ("poly", {"degree": 2, "gamma": 1e-4}),
    ("poly", {"degree": 3, "gamma": 1e-4}),
    ("linear", {})
]

acc_curves = {}
best_acc = -1.0
best_info = None

for kernel, kparams in tqdm(kernel_settings, desc="Kernels"):
    parts = [kernel]
    for k, v in kparams.items():
        parts.append(f"{k}={v}")
    label = ", ".join(parts)

    acc_list = []

    for d in tqdm(dims, desc=f"{label} dims", leave=False):
        kpca = KernelPCA(
            n_components=d,
            kernel=kernel,
            fit_inverse_transform=False,
            random_state=0,
            **kparams
        )
        trainX_k = kpca.fit_transform(trainX)
        testX_k = kpca.transform(testX)

        clf = LogisticRegressionCV(
            Cs=3,
            cv=3,
```

```

        multi_class='multinomial',
        max_iter=3000,
        solver='lbfgs',
        n_jobs=-1
    )
    clf.fit(trainX_k, trainY)

    test_pred = clf.predict(testX_k)
    acc = accuracy_score(testY, test_pred)
    acc_list.append(acc)

    if acc > best_acc:
        best_acc = acc
        best_info = (label, d)

    acc_curves[label] = acc_list

print("Best KPCA setting:")
print(f"  kernel/param = {best_info[0]}")
print(f"  n_components = {best_info[1]}")
print(f"  test accuracy = {best_acc:.3f}")

```

```

Kernels:   0%|          | 0/5 [00:00<?, ?it/s]
rbf, gamma=0.001 dims:   0%|          | 0/100 [00:00<?, ?it/s]
rbf, gamma=0.0001 dims:  0%|          | 0/100 [00:00<?, ?it/s]
poly, degree=2, gamma=0.0001 dims:  0%|          | 0/100 [00:00<?, ?it/s]
poly, degree=3, gamma=0.0001 dims:  0%|          | 0/100 [00:00<?, ?it/s]
linear dims:  0%|          | 0/100 [00:00<?, ?it/s]

Best KPCA setting:
  kernel/param = rbf, gamma=0.0001
  n_components = 24
  test accuracy = 0.950

```

```

[15]: n_kernels = len(acc_curves)

fig, axes = plt.subplots(1, n_kernels, figsize=(4*n_kernels, 4), sharey=True)

if n_kernels == 1:
    axes = [axes]

for ax, (label, acc_list) in zip(axes, acc_curves.items()):
    ax.plot(dims, acc_list, marker='o')
    ax.set_title(label, fontsize=10)
    ax.set_xlabel("#components")

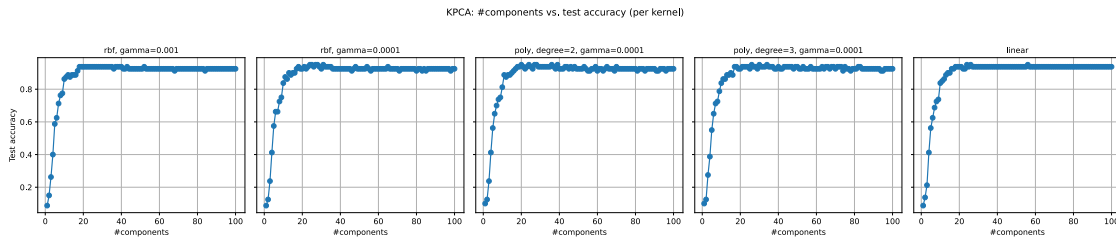
```

```

ax.grid(True)

axes[0].set_ylabel("Test accuracy")
fig.suptitle("KPCA: #components vs. test accuracy (per kernel)", y=1.05)
plt.tight_layout()
plt.show()

```



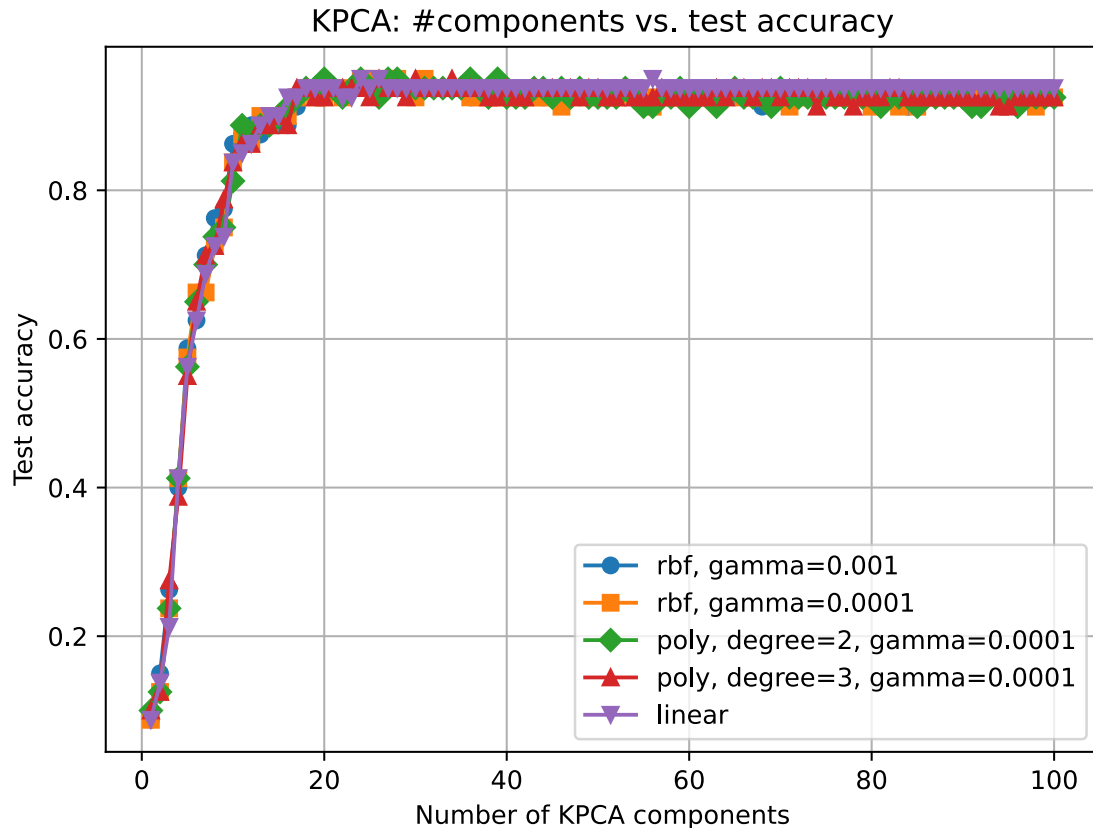
```

[16]: plt.figure(figsize=(7, 5))

marker = ['o', 's', 'D', '^', 'v', 'x', '*', 'P', 'H', '<', '>']
marker_counter = 0
for label, acc_list in acc_curves.items():
    plt.plot(dims, acc_list, marker=marker[marker_counter], label=label)
    marker_counter += 1

plt.xlabel("Number of KPCA components")
plt.ylabel("Test accuracy")
plt.title("KPCA: #components vs. test accuracy")
plt.grid(True)
plt.legend()
plt.show()

```



What is the best kernel and number of components? - **INSERT YOUR ANSWER HERE** - The highest test accuracy is given by the RBF kernel with $\gamma=1e-3$, using 24 components.

Which dimensionality reduction method performs the best for classification? Why? - **INSERT YOUR ANSWER HERE** - The PCA projection performs best: with 51 components it reaches a test accuracy of 0.963, higher than KPCA (0.950) and NMF (0.887). - This means a linear low-dimensional subspace already captures most of the discriminative variance in the data, so PCA both reduces noise and preserves class-separating information better than the nonlinear KPCA mapping or the parts-based NMF representation.

5 De-noising with dimensionality reduction

In this section, we will use dimensionality reduction for denoising a face image. Given a noisy image $\mathbf{x}$, we first compute the low-dimensional representation $\mathbf{w}$, e.g., using PCA. Then given $\mathbf{w}$, we reconstruct the image $\hat{\mathbf{x}}$ from the PCA/NMF basis. In sklearn, the reconstruction step can be achieved using the `inverse_transform()` method of the PCA or NMF classes.

Train a PCA or NMF model using a large number of components (e.g., 100) on the face dataset, and then test it for denoising. For the noisy images, you can try different types of noise, such as: - additive noise (using `random.normal`) - salt & pepper noise (using `random.binomial`) - masking out part of the image (the masked out part can be replaced with black, gray, or white).

for evaluation of the reconstruction or denoising result you can use “peak signal to noise ratio”, which is implemented in skimage as `skimage.metrics.peak_signal_noise_ratio` or `skimage.measure.compare_psnr` (on older versions).

```
[ ]: import skimage
```

```
[19]: ### INSERT YOUR CODE HERE
import numpy as np
from sklearn.decomposition import PCA, NMF
from skimage.metrics import peak_signal_noise_ratio as psnr

n_components = 100

pca = PCA(n_components=n_components, random_state=0)
pca.fit(trainX)

nmf = NMF(n_components=n_components, init="nndsvda", random_state=0,
    ↪max_iter=500)
nmf.fit(trainX)

rng = np.random.RandomState(0)

sigma = 0.20
X_test_noisy = testX + sigma * rng.normal(size=testX.shape)
X_test_noisy = np.clip(X_test_noisy, 0.0, 1.0)

prob_sp = 0.05
X_test_noisy_sp = testX.copy()
mask = rng.binomial(1, prob_sp, size=testX.shape)
X_test_noisy_sp[mask == 1] = rng.binomial(1, 0.5, size=testX[mask == 1].shape)

[20]: X_pca_rec = pca.inverse_transform(pca.transform(X_test_noisy))

X_nmf_rec = nmf.inverse_transform(nmf.transform(X_test_noisy))

def mean_psnr(X_clean, X_rec, image_shape, data_range=1.0):
    psnrs = []
    for x0, xhat in zip(X_clean, X_rec):
        psnrs.append(
            psnr(x0.reshape(image_shape),
                  xhat.reshape(image_shape),
                  data_range=data_range)
        )
    return np.mean(psnrs)

avg_psnr_pca = mean_psnr(testX, X_pca_rec, imgsize)
avg_psnr_nmf = mean_psnr(testX, X_nmf_rec, imgsize)
```

```

print(f"Average PSNR (PCA, Gaussian noise) = {avg_psnr_pca:.2f} dB")
print(f"Average PSNR (NMF, Gaussian noise) = {avg_psnr_nmf:.2f} dB")

import matplotlib.pyplot as plt

n_show = 5
fig, axes = plt.subplots(3, n_show, figsize=(2*n_show, 6))

for i in range(n_show):
    axes[0, i].imshow(testX[i].reshape(imgsize), cmap="gray")
    axes[0, i].axis("off")
    axes[0, i].set_title("Clean")

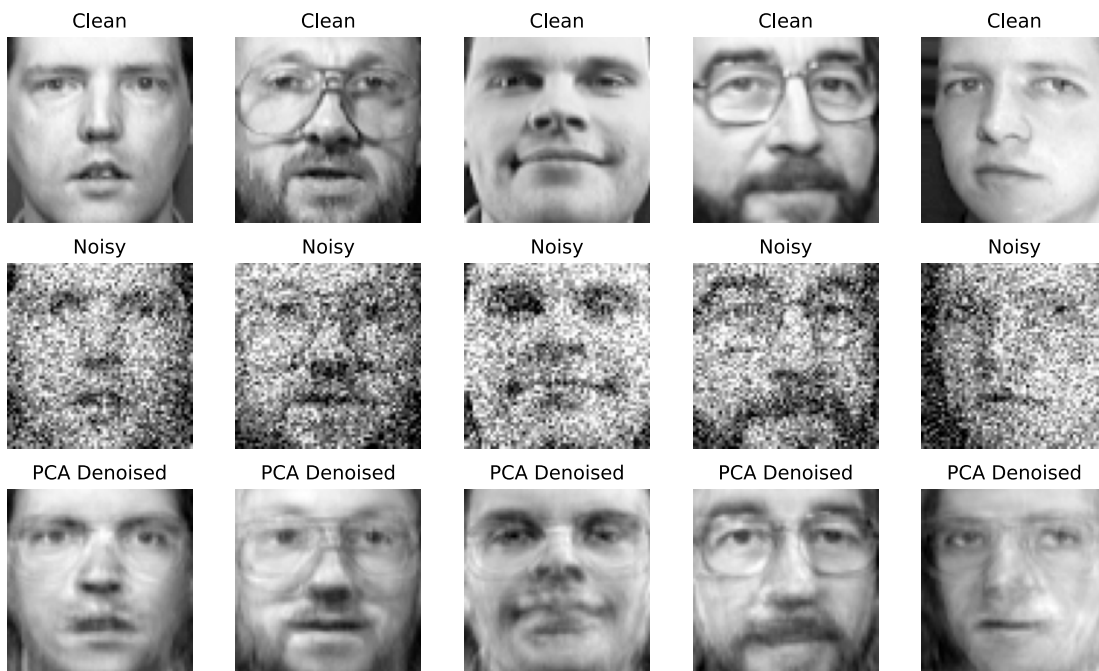
    axes[1, i].imshow(X_test_noisy[i].reshape(imgsize), cmap="gray")
    axes[1, i].axis("off")
    axes[1, i].set_title("Noisy")

    axes[2, i].imshow(X_pca_rec[i].reshape(imgsize), cmap="gray")
    axes[2, i].axis("off")
    axes[2, i].set_title("PCA Denoised")

plt.tight_layout()
plt.show()

```

Average PSNR (PCA, Gaussian noise) = 24.66 dB
Average PSNR (NMF, Gaussian noise) = 24.65 dB



```

[21]: X_pca_rec = pca.inverse_transform(pca.transform(X_test_noisy_sp))

X_nmf_rec = nmf.inverse_transform(nmf.transform(X_test_noisy_sp))

def mean_psnr(X_clean, X_rec, image_shape, data_range=1.0):
    psnrs = []
    for x0, xhat in zip(X_clean, X_rec):
        psnrs.append(
            psnr(x0.reshape(image_shape),
                 xhat.reshape(image_shape),
                 data_range=data_range)
        )
    return np.mean(psnrs)

avg_psnr_pca = mean_psnr(testX, X_pca_rec, imgsize)
avg_psnr_nmf = mean_psnr(testX, X_nmf_rec, imgsize)

print(f"Average PSNR (PCA, Salt-and-Pepper noise) = {avg_psnr_pca:.2f} dB")
print(f"Average PSNR (NMF, Salt-and-Pepper noise) = {avg_psnr_nmf:.2f} dB")

import matplotlib.pyplot as plt

n_show = 5
fig, axes = plt.subplots(3, n_show, figsize=(2*n_show, 6))

for i in range(n_show):
    axes[0, i].imshow(testX[i].reshape(imgsize), cmap="gray")
    axes[0, i].axis("off")
    axes[0, i].set_title("Clean")

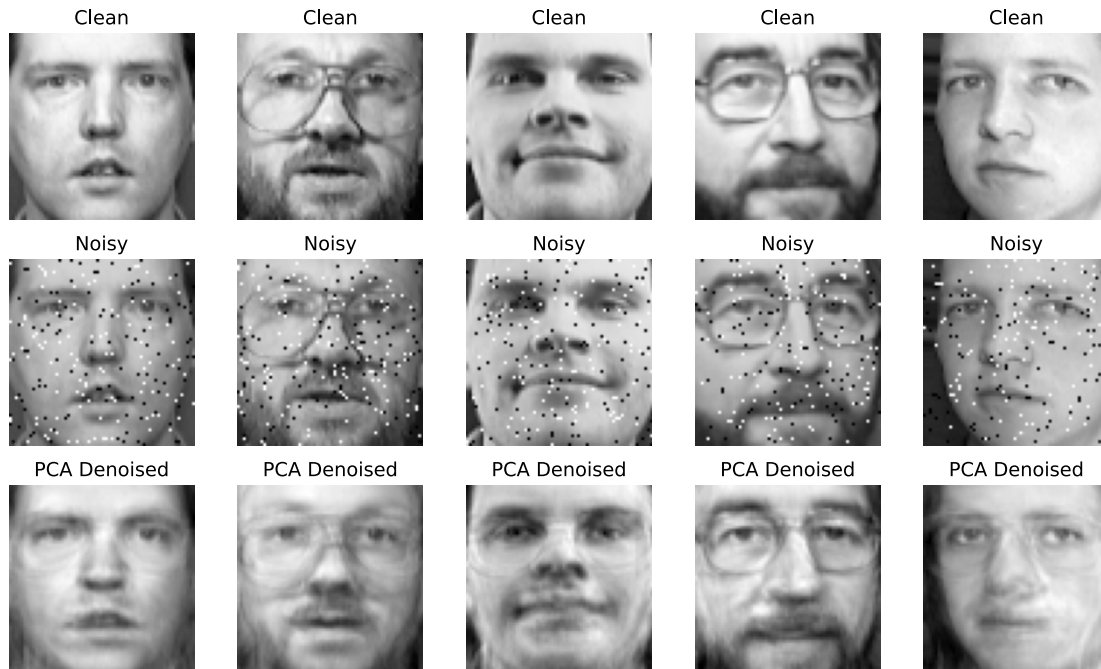
    axes[1, i].imshow(X_test_noisy_sp[i].reshape(imgsize), cmap="gray")
    axes[1, i].axis("off")
    axes[1, i].set_title("Noisy")

    axes[2, i].imshow(X_pca_rec[i].reshape(imgsize), cmap="gray")
    axes[2, i].axis("off")
    axes[2, i].set_title("PCA Denoised")

plt.tight_layout()
plt.show()

```

Average PSNR (PCA, Salt-and-Pepper noise) = 25.42 dB
 Average PSNR (NMF, Salt-and-Pepper noise) = 25.02 dB



Which kind of noise can be handled well by linear dimensionality reduction? Why? - **INSERT YOUR ANSWER HERE** - Linear dimensionality reduction is good at removing random, small-magnitude additive noise (both zero-mean Gaussian and sparse salt-and-pepper noise). Faces lie roughly in a low-dimensional linear subspace, while this kind of noise is high-dimensional and spread almost independently over pixels. Projecting onto the top components keeps the low-dimensional face subspace and discards directions with only noise, so the reconstruction averages out the noise and preserves the underlying image.